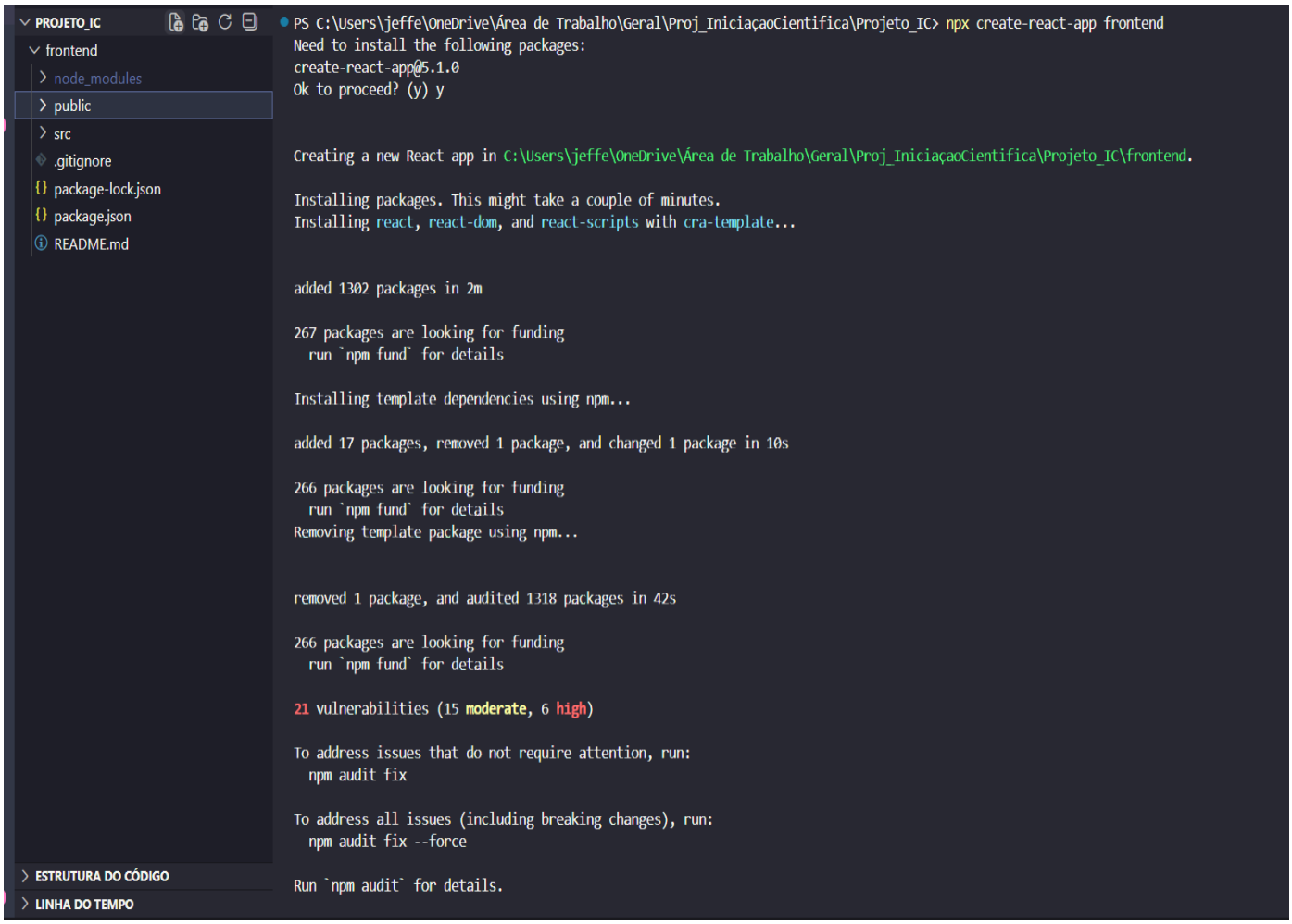
Versões utilizadas no projeto (fase atual):

- Node.js: v20.20.0
- React: v19.2.4

Data: 02/02

Para começar, criei a pasta Projeto_IC, entrei no terminal e digitei:

npx create-react-app frontend



Depois, entrei na pasta que criei (**cd frontend**) e rodei o comando **npm star**t para abrir a aplicação no navegador em localhost:3000.

OBS: {Sobre o arquivo **src/reportWebVitals.js**, ele não é essencial para rodar a aplicação React. Porém, esse arquivo é útil se quisermos monitorar a performance real da aplicação em produção, coletando métricas de experiência do usuário. Decidi manter ele, falar com o Evandro depois para decidir se vamos utilizá-lo.}

Dentro da pasta **src**, criei a pasta **components**. Dentro dela, criei a pasta **layout** e, dentro dessa pasta, o arquivo **MainLayout.jsx**.

OBS: {Hoje, em aplicações React, não existe mais um arquivo .html separado para cada tela. Em vez disso, utiliza-se o JSX, que nada mais é do que JavaScript capaz de retornar algo muito parecido com HTML}

Dentro do MainLayout.jsx, comecei importando o **Sidebar.jsx**, que será o nosso menu lateral. Também utilizei algo chamado **children ({children})** dentro da tag . Esse children funciona como um espaço vazio que será preenchido pelo App.js (ou qualquer outro componente que use o MainLayout). Tudo o que for escrito dentro de ... aparece exatamente no lugar do {children}.

Data: 03/02

`O App.js é o orquestrador da aplicação.`

Depois de criar o menu lateral, antes de partir para o menu superior, precisei deixar as opções (como “Zero de Funções”) clicáveis e armazenar essa escolha em uma variável.

Pensei inicialmente em guardar esse estado no Sidebar.jsx, mas percebi que o ideal é manter o estado no App.js, já que o estado deve ficar no nível mais alto que precisa dele.

No momento, o CSS do Sidebar está dentro do próprio arquivo Sidebar.jsx. Mais para frente, pretendo criar um arquivo Sidebar.css separado apenas para estilização.

Data: 04/02

Relacionando com o do dia anterior:

import { useState } from "react";

O import { useState } from "react" traz apenas o hook useState da biblioteca React para dentro do arquivo. O useState permite criar variáveis de estado dentro de componentes funcionais, ou seja, valores que podem mudar ao longo do tempo e que, quando atualizados pela função correspondente (ex: setCategoriaSelecionada), fazem o componente ser renderizado novamente automaticamente.

O useState(null) retorna um array com dois itens:

1. **O valor atual do estado (inicialmente null).**
2. **A função que atualiza esse valor.**

A linha:

const [categoriaSelecionada, setCategoriaSelecionada] = useState(null)

utiliza desestruturação do JavaScript para separar esses dois valores.

- Ele pega o **primeiro item** do array e coloca em `categoriaSelecionada`.

- Ele pega o **segundo item** do array e coloca em `setCategoriaSelecionada`.

Antes, o Sidebar estava assim:

```
<Sidebar />
```

OBS: Ou seja apenas renderizava o menu lateral. Depois que criei a lógica de categoria, passei uma prop para o Sidebar:

Agora que criei a categoria

Depois que criei a lógica de categoria, passei uma prop para o Sidebar:

```
<Sidebar aoSelecionarcategoria={aoSelecionarcategoria} />
```

Porém o MainLayout não tem acesso direto ao estado. então precisei passar essa função primeiro para o MainLayout:

```
<MainLayout aoSelecionarcategoria={setCategoriaSelecionada}>
```

E depois recebê-la como parâmetro no MainLayout.jsx:

function MainLayout({ *children* , *aoSelecionarcategoria*}{...

OBS: Componentes filhos não alteram diretamente o estado global. Eles apenas avisam que algo aconteceu, como: “alguém clicou aqui”.

No arquivo Sidebar.jsx, passei a receber essa função:

```
function Sidebar({ aoSelecionarcategoria }) {
```

Assim, o Sidebar passa a conhecer a função que veio do App.js, que é o nosso orquestrador.

Em cada item do menu lateral, adicionei um onClick para chamar essa função, passando o nome da categoria selecionada:

Depois disso, comecei a planejar o menu superior, que só deve aparecer quando o usuário selecionar uma opção no menu lateral.

> tenho que criar um arquivo menu superior. jsx no mesmo caminho que os outros, fazer ele receber a categoria selecionada. (var categoriaselecionad))
>
> e daí adicionar alguns botões sobre cada método que pode ser feito para aquela conta(exemplo: usuário selecionou zero de funções no menu lateral) vai abrir o menu superior com dois botões com Bisseção e Newton ele apertando em algum botão. A tela abaixo irá…?

Data: 06/02

Criei o arquivo **TopMenu.jsx** só que diferente do **sidebar.jsx (menulateral)** eu nao instanciei ele dentro do **mainLayout.jsx** importando ele no começo.

Eu adicionei ele dentro da tag **<main Layout>** dentro do **app.js** pelo fato que o topmenu nao vai ser algo fixo na tela, igual o menu lateral.

OBS: Acredito que pensei certo nisso. **Esta?**

OBS:

- O Sidebar **sempre existe**
- Ele faz parte da **estrutura da aplicação**
- Independente do que o usuário escolher o Sidebar estará lá

Dentro do **topmenu** comecei criando um **IF** para ver qual categoria foi selecionada no **sidebar**.

Dentro do TopMenu, comecei criando uma condição para verificar qual categoria foi selecionada no Sidebar. Inicialmente implementei apenas para “Zero de Funções”, exibindo dois botões: Bisseção e Newton.

Após isso vou colocar as verificações para integrais e interpolação, adicionar os botões para integrais e interpolação e pretendo implementar as categorias de Interpolação e Integrais e, por fim, organizar melhor o CSS do projeto.

OBS:

Sidebar seleciona Categoria
TopMenu seleciona Método
App decide o que vai renderizar

Dúvidas matemáticas:

Interpolação

A interpolação serve para encontrar valores intermediários entre pontos conhecidos.

> **Polinômios de Lagrange:** É a forma clássica de ajustar um polinômio que passa por todos os pontos. Muito didático, mas pode sofrer com oscilações nas bordas (Fenômeno de Runge).
>
> **Splines Cubícolas:** De longe a mais utilizada na prática. Em vez de um único polinômio complexo, ela usa vários polinômios de grau 3 conectados. É o que softwares de design e animação usam para criar curvas suaves.

Polinômios de Newton: Excelente quando você precisa adicionar mais pontos ao conjunto de dados sem ter que refazer todo o cálculo do zero, graças às

Diferenças Divididas (?).

Integrais

Integração Numérica (Onde a conta de verdade acontece)

Basicamente, a gente usa isso quando a integral é muito difícil.

Regra do Trapézio: É a mais "feijão com arroz". Você finge que o gráfico é um monte de trapézios e soma a área deles. Não é a mais precisa do mundo (ela corta as curvas com retas, então perde um pouco de informação)

Regra de Simpson (1/3 e 3/8): Essa aqui em vez de ligar os pontos com retas, ela usa curvas (parábolas). E com quase o mesmo esforço do trapézio, conseguimos um resultado muito mais próximo do real.

Quadratura Gaussiana: Enquanto nas outras usamos pontos com a mesma distância entre eles, aqui escolhe pontos estratégicos (que não são óbvios) para chegar no resultado com muito menos conta.

Data: 10/03 e 11/03

Antes eu só tinha feito (teste) com o “Zero de funções” e aparecia dois botões: “Bisseção” e “Newton”.

Agora fiz uma melhoria para aparecer os métodos da categoria selecionada pelo usuário, para isso a minha ideia foi em vez de colocar vários IFs, fiz uma breve estrutura (Objeto) que daí associa cada categoria ao seus métodos.

```
const metodosDisponiveis = {
    "Zero de Funções": ["Bisseção", "Newton"],
    "Interpolação": ["Lagrange", "Newton"],
    "Integrais": ["Trapézio", "Simpson"]
}
```

Assim, quando a categoria é selecionada, o sistema busca no objeto os métodos correspondentes. Caso não existir não exibe nada

pra isso eu utilize a função .map() que vai passar por todos os elementos do nosso array (vetor metodosDaCategoria) e caso a gente precisar adicionar mais métodos fica bem fácil.

Data: 12/03 e 13/03

Agora vou criar, dentro da pasta src/, a pasta chamada pages. Nela, vamos criar um arquivo para cada método, onde cada tela será diferente. Acho que vou começar pelo Bisseção.jsx, já que é um dos mais simples e esta como exemplo no documento de protótipo.

Nós já temos uma variável que guarda o método que o usuário selecionou (metodoSelecionado), agora no App.js vou importar bissecao.jsx.

```
import Bissecao from ".pages/Bissecao";
```

E vamos usar o MetodoSelecionado para renderizar a página abaixo correta para cada método.

Antes como teste eu tinha apenas um print do metodoSelecionado para verificar se estava realmente “pegando” o método certo. Agora para renderizar a página abaixo eu fiz uma breve verificação.

```
{metodoSelecionado === "Bissecao" && <Bissecao />}
```

Se o método que o usuário selecionou for o bissecao (que é o único por enquanto) vai renderizar o arquivo bissecao.jsx

No momento nossa estrutura do projeto está assim:

SRC/

components/

layout/

Sidebar.jsx (nosso menu lateral)

TopMenu.jsx (nosso menu superior com os métodos da categoria que o usuario selecionou)

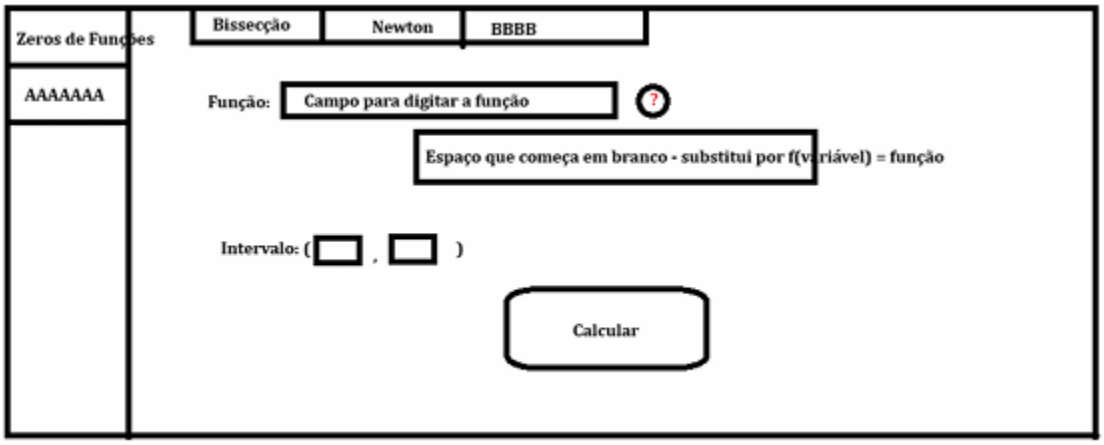
MainLayout.jsx (onde chamamos o sidebar)

pages/

Bissecao.jsx (recém criado)

Data: 16/03

Agora eu vou implementar algo no bissecao.jsx para quando o usuário selecionar esse método aparecer as opções. Vou me basear brevemente nessa imagem aqui:



se o usuário apertar o método Bissecao vai aparecer um campo para ele digitar f(x) = (x^2 - 4)

a = 1
b = -6

e um botão de calcular.

Para fazer isso comecei com algo bem básico, apenas criei 3 input para o usuário digitar. Depois tenho que melhorar o CSS.

Data: 27/03

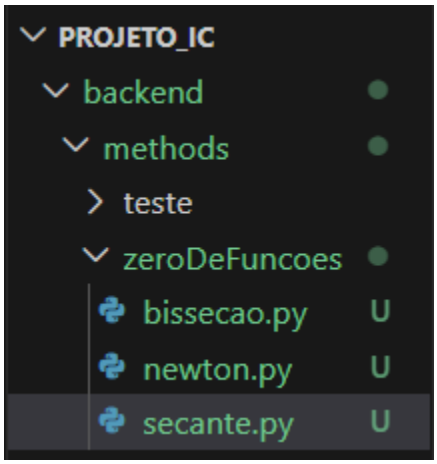
Eu e o professor Evandro construímos algumas das contas que vamos utilizar no projeto na parte de back-end.

Com esse arquivo pronto, criei uma pasta chamada backend (no mesmo nível da pasta frontend). Dentro da pasta backend, criei um arquivo chamado main.py, onde acredito que vamos centralizar as chamadas das funções e possivelmente utilizar uma API.

Também criei uma pasta chamada methods e, dentro dela, uma outra pasta chamada ZeroDeFuncoes. Dentro dessa pasta, criei três arquivos: bissecao.py, newton.py e secante.py, separando cada método em seu próprio arquivo para organizar melhor o projeto.

Além disso, criei um arquivo chamado utils.py dentro da pasta ZeroDeFuncoes. Nele, definimos a variável x utilizando a biblioteca Sympy e uma função auxiliar. Esse arquivo é essencial porque é ele que permite transformar a função simbólica em um valor numérico quando substituímos um valor em x. Sem isso, o programa não consegue realizar os cálculos corretamente.

ESTRUTURA BREVE:



Data: 01/04 a 13/04

Eu e o professor Evandro realizamos a integração entre o front-end (React) e o back-end (Python com FastAPI).

Agora, quando o usuário preenche os valores no método da Bisseção (função, intervalo e critério) e clica em “Calcular”, a aplicação envia esses dados para o back-end, que realiza o cálculo e retorna o resultado para o front-end.

Essa comunicação foi feita utilizando uma API criada com FastAPI.

No arquivo main.py, comecei criando a aplicação com FastAPI:

app = FastAPI()

Depois disso, adicionei uma configuração chamada CORS (Cross-Origin Resource Sharing). Isso é necessário porque o front-end (rodando no localhost:3000) e o back-end (rodando em outra porta, como 8000) são considerados origens diferentes pelo navegador. Sem isso, o React não conseguiria acessar o Python.

```
app.add_middleware(
CORSMiddleware,
allow_origins=["*"],
allow_methods=["*"],
allow_headers=["*"],
)
```

Em seguida, criei uma classe chamada BissecaoInput utilizando Pydantic. Essa classe define o formato dos dados que o back-end espera receber do front-end (função, valores de a e b, e critério).

Depois disso, criei uma função auxiliar chamada calcular_f, responsável por transformar a função digitada pelo usuário (string) em uma expressão matemática utilizando Sympy, e calcular seu valor para um determinado x.

A principal parte foi a criação da rota:

@app.post("/calcular")

Essa rota recebe os dados enviados pelo front-end, executa o método da Bisseção e retorna o resultado em formato JSON.

Dentro dessa função:

- Primeiro, verifico se existe raiz no intervalo informado (f(a) * f(b) > 0)
- Depois, aplico o método da Bisseção utilizando um loop

- Armazenei cada iteração em uma lista (iteracoes), contendo o número da iteração, o valor médio e o valor da função
- No final, retorno a raiz encontrada e todas as iterações

O retorno é feito assim:

```
{
"raiz": valor,
"iteracoes": [...]
}
```

Com isso, o front-end agora consegue consumir os dados do back-end e mostrar o resultado para o usuário.

No momento, essa integração foi feita apenas para o método da Bisseção, mas a ideia é expandir para os outros métodos posteriormente.

Após integrar o front-end com o back-end, fiz melhorias no arquivo Bissecao.jsx para permitir que o usuário interaja de forma completa com o sistema.

Comecei criando estados (useState) para armazenar os valores digitados pelo usuário, como a função, os limites a e b, e o critério de parada. Também defini valores iniciais para facilitar os testes.

Além disso, criei estados para armazenar o retorno da API (resultado), possíveis erros (erro) e um estado de carregamento (carregando), que indica quando a requisição está sendo processada.

Depois disso, implementei a função handleCalcular, que é responsável por enviar os dados para o back-end. Para isso, utilizei o fetch, fazendo uma requisição do tipo POST para a rota /calcular.

Nessa requisição, envio os dados em formato JSON (função, valores de a, b e critério). Também utilizei parseFloat para garantir que os valores numéricos sejam enviados corretamente.

Após receber a resposta do servidor, faço uma verificação:

- Se houver erro retornado pela API, ele é exibido na tela
- Caso contrário, o resultado é armazenado no estado e exibido para o usuário

Também tratei possíveis erros de conexão, exibindo uma mensagem caso o back-end não esteja rodando.

Na interface, adicionei:

- Campos de entrada para o usuário digitar os valores
- Um botão de calcular, que muda para “Calculando...” enquanto a requisição está em andamento
- Exibição da raiz aproximada encontrada
- Exibição do número de iterações
- Uma tabela mostrando cada iteração do método da Bisseção (índice, valor médio e f(x))

Com isso, o usuário agora consegue não apenas enviar os dados, mas também visualizar o passo a passo do método, tornando a aplicação mais didática e interativa.

Data: 16/04 e 17/04

Backend:

Criei mais uma pasta na pasta methods chamada sistemasLineares, onde adicionei dois métodos (jacobi e Seidel). Dentro da pasta sistemasLineares tambem adicionei o arquivo utils.py que é os criterios da função (Igual o bissecção tem o dele).

no main.py fiz algumas breves mudanças, importei esses dois novos arquivos no começo do main. E também adicionei as rotas para as funções de sistemasLineares.

Frontend:

Adicionei no sideBar.jsx essa linha aqui:

```
        <li style={{cursor: "pointer", marginBottom: "10px"}} onClick={()
=> aoSelecionarcategoria("Sistemas Lineares")}>Sistemas Lineares</li>
```

Assim permite o usuário selecionar o método sistemas Lineares e dentro dele tera dois métodos possíveis (jacobi e Seidel).

E no TopMenu.jsx adicionei em MetodosDisponiveis:

"Sistemas Lineares": ["Jacobi", "Gauss-Seidel"],

Que são os “botões” os métodos que vamos ter disponíveis para Sistemas Lineares

E no nosso App.js adicionei esses dois métodos possíveis que iram chamar nossos arquivos jacobi.jsx (react) e Seidel.jsx (react). Esses dois arquivos reacts vou criar ainda.

Data: 21/04

Nesta etapa do projeto, realizamos uma evolução importante na integração entre o front-end e o back-end, além de melhorias na visualização dos resultados.

No back-end, foram realizados ajustes no arquivo main.py, mantendo a comunicação com o front-end via API utilizando o FastAPI.

No front-end, foi criada uma nova estrutura de componentes para melhorar a organização do código. Dentro da pasta components, foi criada uma subpasta chamada bisseccao, onde foi adicionado o componente CardIteracaoBissecao.jsx. Esse componente é responsável por exibir de forma visual e interativa cada iteração do método da bisseção.

Além disso, o arquivo Bissecao.jsx foi atualizado para consumir esse novo componente, permitindo que, após o cálculo, sejam exibidos cards detalhando cada passo do método, incluindo valores de a, b, média e o comportamento da função.

Também foi adicionada uma visualização gráfica utilizando a biblioteca Chart.js, permitindo ao usuário acompanhar visualmente a convergência da raiz ao longo das iterações.

Essas melhorias tornam o sistema mais didático e interativo, facilitando o entendimento do método numérico.

Data: 25 a 27/04

Nessa etapa do projeto, eu e o professor fizemos melhorias importantes no método da bisseção, principalmente na parte visual e na forma como o usuário interage com o sistema.
A principal mudança foi a criação de diferentes níveis de detalhamento para o cálculo. Agora o usuário pode escolher entre quatro modos: Básico, Intermediário, Completo e Interativo. Cada modo muda a forma como as informações são exibidas na tela.

No modo Básico, o sistema mostra apenas o resultado final, sem muitos detalhes.

No modo Intermediário, já são exibidas algumas informações do processo e o grafico, como valores das iterações.

No modo Completo, o usuário consegue ver todos os dados envolvidos no cálculo, incluindo todas as iterações realizadas.

E já o modo Interativo foi o mais interessante, pois ele mostra o cálculo acontecendo aos poucos, passo a passo, facilitando muito o entendimento do método.
Além disso, também melhoramos bastante a parte visual. Foi adicionado um gráfico que mostra os pontos a, b e a média, ajudando a entender como o intervalo vai diminuindo até chegar na raiz.

Para organizar melhor o código, criamos um novo componente dentro da pasta components/bisseccao, chamado CardIteracaoBissecao.jsx. Esse componente é responsável por mostrar cada iteração de forma visual, deixando o sistema mais organizado e mais fácil de manter. Essas mudanças deixaram o projeto mais didático e mais fácil de usar, principalmente para quem está aprendendo **métodos numéricos**.

Data: 28/04

Nessa etapa do projeto, fizemos melhorias na organização e no padrão visual da interface.

Foi criada uma nova pasta chamada UI dentro de components, com o objetivo de centralizar elementos visuais reutilizáveis. Dentro dessa pasta, criamos o arquivo Botao.jsx, que define um padrão para os botões utilizados no sistema.

A ideia foi evitar repetição de código e manter um visual mais consistente em toda a aplicação.

Além disso, atualizamos o arquivo TopMenu.jsx, substituindo os botões antigos pelos novos botões padronizados. Com isso, todos os métodos passaram a utilizar o mesmo estilo de botão, deixando a interface mais organizada e uniforme.

Essa mudança não altera a lógica do sistema, mas melhora a estrutura do código e a experiência visual do usuário.

Data: 29/04

Nessa etapa do projeto, foquei principalmente em melhorar a parte técnica do backend, deixando o sistema mais seguro, correto e organizado.

Uma das primeiras coisas que foi corrigida foi uma falha grave de segurança. O sistema utilizava o sympy.parse_expr sem restrições, o que permitia que alguém mal-intencionado executasse comandos no servidor. Isso foi corrigido limitando o que pode ser interpretado, permitindo apenas funções matemáticas seguras e bloqueando qualquer código perigoso. Além disso, também foi adicionada uma validação para evitar esse tipo de problema antes mesmo de processar a expressão.

Outro ponto importante foi um erro no método de Newton. A derivada estava sendo calculada de forma incorreta, o que poderia gerar resultados errados sem aviso. Isso foi ajustado especificando corretamente a variável na função de derivação, garantindo que o cálculo seja feito da forma certa.

Também foi feita uma limpeza geral no código. Existiam trechos duplicados, código comentado e até versões diferentes da mesma função dentro dos arquivos. Tudo isso foi reorganizado, deixando apenas uma versão correta de cada método (bisseção, Newton e secante). Isso deixou o código muito mais limpo e fácil de entender.

Além disso, a estrutura do backend foi melhorada. Antes, a lógica matemática estava misturada dentro do main.py. Agora, o main.py ficou responsável apenas pelas rotas, enquanto os cálculos foram separados em arquivos próprios. Isso facilita bastante a manutenção e organização do projeto.

Um avanço muito importante foi a criação de testes automatizados usando pytest. Antes não existia nenhum teste, o que tornava difícil garantir que tudo estava funcionando corretamente. Agora foram criados 44 testes cobrindo vários cenários, como casos simples, erros

matemáticos e validação de entrada. Todos os testes passaram, o que dá mais segurança para continuar evoluindo o projeto.

Por fim, também foi ajustada a forma como o sistema interpreta expressões matemáticas, garantindo que funcione corretamente sem comprometer a segurança.
No geral, essas mudanças fizeram o backend evoluir bastante, saindo de algo mais experimental para uma base mais confiável e pronta para crescer.

Data: 30/04

Nesse dia, o foco foi melhorar a arquitetura do sistema, principalmente a comunicação entre frontend e backend.
Foi identificado um problema importante: a lógica do método da bisseção estava sendo executada várias vezes no frontend. Ou seja, o mesmo cálculo era feito em diferentes arquivos, o que não é uma boa prática.

Por exemplo:

> Em Bissecao.jsx, o cálculo era feito usando new Function()
> Em BissecaoInterativa.jsx, existia outra função que fazia o cálculo novamente
> Em CardIteracaoBissecao.jsx, ainda havia mais uma forma de calcular valores para o gráfico

Isso fazia com que o mesmo cálculo fosse executado três vezes para a mesma entrada. Além disso, cada parte usava uma forma diferente de calcular, o que podia gerar resultados inconsistentes.

Outro problema sério era o uso de new Function() no frontend, que também representa um risco de segurança, já que permite executar código a partir de texto sem validação.
Para resolver isso, foi feita uma mudança importante na arquitetura do projeto.

Agora, todo o cálculo é feito exclusivamente no backend, que passou a ser a única fonte de verdade. O frontend deixou de calcular e passou apenas a enviar os dados para o backend e exibir os resultados.

O backend agora possui endpoints específicos para cada método:

> /calcular para bisseção
> /newton
> /secante

E retorna os dados já prontos, incluindo a raiz e todas as iterações.
No frontend, várias mudanças foram feitas:

No arquivo Bissecao.jsx, foi removida toda a lógica de cálculo. Agora ele apenas coleta os dados do usuário, chama o backend e envia o resultado para o componente interativo.

No BissecaoInterativa.jsx, também foi removida a parte de cálculo. Em vez disso, ele passou a usar os dados recebidos do backend para montar os passos do método, mantendo toda a parte pedagógica (como perguntas, dicas e interações com o usuário).

Já o CardIteracaoBissecao.jsx deixou de calcular valores e passou a usar apenas os dados já processados.

Outra melhoria importante foi a padronização matemática. Antes, havia inconsistência no uso de funções como log (às vezes log natural, às vezes log base 10). Agora tudo segue o padrão do backend, que utiliza log natural.

A única exceção é a calculadora do sistema, que continua usando graus para seno e cosseno. Isso foi mantido de propósito, pois ela serve apenas como apoio para o aluno, não interferindo no cálculo principal.

Também foram criadas duas novas pastas no frontend:

api/, responsável pelas chamadas ao backend
hooks/, onde foi criado o useBissecao para organizar melhor a lógica de requisições

Essas mudanças melhoraram bastante a organização do código e preparam o projeto para crescer, facilitando a implementação futura de outros métodos como Newton e Secante no modo interativo.

No geral, essa etapa foi muito importante, pois deixou o sistema mais organizado, seguro e alinhado com boas práticas de desenvolvimento.

11/05/2026

Frontend

- TopMenu.jsx — corrigido bug onde "secante" estava em minúsculo, fazendo a página não renderizar ao clicar no botão (a comparação em App.js esperava "Secante").
- Secante.jsx — reescrito completamente com tema claro (fundo branco, cor roxa #3c3489), seguindo o mesmo padrão visual da Bisseção. Adicionados os campos x_0 e x_1, critério de parada configurável e seletor de detalhamento (Básico / Intermediário / Completo).

- Newton.jsx — reescrito com tema claro, igualando o padrão visual da Bisseção e Secante. Removido o tema escuro anterior.

Backend

- utils.py — resolvido bug crítico de incompatibilidade do SymPy com Python 3.14. A função F() parou de usar evalf() e lambdify, passando a converter a expressão SymPy para string e avaliá-la com eval() usando um namespace restrito com funções do módulo math. O SymPy continua sendo usado apenas para parsear e derivar.
- newton.py — atualizada assinatura: inicial → x0, adicionados tolerancia, criterio (string) e max_iter. Iterações agora incluem os campos dfx, x_novo e erro. Avaliação da derivada passou a usar F().
- secante.py — atualizada assinatura: a/b → x0/x1, adicionados tolerancia, criterio e max_iter. Campos das iterações renomeados para x_anterior, x, fx_anterior, fx, x_novo, erro, compatíveis com o frontend.
- main.py — schemas EntradaNewton e EntradaSecante atualizados para refletir os novos campos enviados pelo frontend.

Data: 12/05/2026

Nessa etapa do projeto, o foco foi expandir o sistema com novos recursos pedagógicos para todos os métodos. Foram feitas várias implementações importantes: correção de um bug crítico no React, adição do modo Interativo para Newton e Secante, implementação completa dos modos Básico, Intermediário, Completo e Interativo para Jacobi e Gauss-Seidel, e implementação do método de Interpolação de Lagrange do zero.

Correção de bug — Crash no React ao calcular com campos vazios (Newton e Secante)
Foi identificado um bug que causava um crash na interface ao tentar calcular sem preencher os campos. O erro exibido era:
Objects are not valid as a React child (found: object with keys {type, loc, msg, input})

O problema estava nos hooks useNewton.js e useSecante.js. Quando o backend recebe dados inválidos, o FastAPI retorna um erro HTTP 422 onde o campo detail é um array de objetos no formato Pydantic (com chaves type, loc, msg, input), e não uma string simples. O hook tentava renderizar esse array diretamente no JSX, causando o crash.

A solução foi feita em dois níveis. Primeiro, adicionamos uma validação antecipada nas páginas Newton.jsx e Secante.jsx: antes de qualquer requisição ao backend, a função validar() verifica se todos os campos estão preenchidos e exibe uma mensagem amigável caso não estejam, sem chegar a acionar o servidor. Segundo, os hooks foram atualizados para tratar qualquer formato de erro da API, verificando se data.detail é uma string, um array ou outro tipo antes de exibir a mensagem. Esse padrão foi adotado como padrão do projeto e aplicado em todos os hooks criados depois.

Modo Interativo — Newton-Raphson e Secante
Seguindo o modelo já existente do BissecaoInterativa.jsx, implementamos os modos interativos para Newton e Secante.

Para o Newton, criamos o arquivo components/newton/NewtonInterativo.jsx. Cada iteração vira uma fase com 4 subpassos que o aluno responde um a um: calcular $f(x_{\square})$, calcular $f'(x_{\square})$, calcular o novo $x_{\square+1}$ pela fórmula, e calcular o erro conforme o critério escolhido (absoluto, relativo ou pela função).

Para a Secante, criamos o arquivo components/secante/SecanteInterativa.jsx. Como o método usa dois pontos ao invés de derivada analítica, os subpassos são 5 por iteração: calcular $f(x_{\square-1})$, calcular $f(x_{\square})$, calcular o denominador $f(x_{\square}) - f(x_{\square-1})$ (subpasso extra para reforçar que divisão por zero encerra o método), calcular $x_{\square+1}$ pela fórmula e calcular o erro.

Ambos os componentes seguem o mesmo padrão visual da Bisseção: calculadora embutida, sistema de dicas em dois níveis (parcial na primeira tentativa errada, completa na segunda), painel de histórico com todas as respostas confirmadas e barra de progresso no topo.
Sistemas Lineares — Jacobi e Gauss-Seidel com todos os modos
Já tínhamos criado na sessão anterior as páginas Jacobi.jsx e GaussSeidel.jsx com os modos Básico, Intermediário e Completo. Nessa sessão concluímos os modos Interativos para os dois métodos.

A interface de entrada dos dois métodos funciona da seguinte forma: o usuário escolhe o tamanho do sistema (de 2×2 até 5×5), preenche a matriz A (com a diagonal destacada visualmente com borda roxa), o vetor b, o chute inicial x^0 e a tolerância. Também adicionamos o componente AvisoDomDiag, que verifica automaticamente se a matriz é estritamente diagonal dominante e exibe um aviso visual caso não seja, informando que a convergência não está garantida.

Para o modo interativo do Jacobi (JacobiInterativo.jsx), cada iteração é uma fase com n subpassos — um por variável x_i. O painel explicativo reforça o conceito-chave do método: todos

os valores usados no cálculo de x_i pertencem à iteração anterior, nenhum valor atualizado é reaproveitado dentro da mesma iteração.

Para o modo interativo do Gauss-Seidel (GaussSeidelInterativo.jsx), a estrutura é igual, mas com a diferença fundamental do método: ao calcular x_i, os valores x_1, ..., x_{i-1} já calculados nessa mesma iteração são usados imediatamente. Para tornar isso visível, adicionamos o símbolo ↑ ao lado de cada valor já atualizado. O painel explicativo de cada subpasso lista os valores com ↑ disponíveis naquele momento, deixando claro para o aluno qual é a diferença em relação ao Jacobi.

OBS: No Jacobi, todos os x□ usados são da iteração anterior. No Gauss-Seidel, os x□ com j < i já foram atualizados nessa mesma iteração e são usados na hora. Isso faz o Gauss-Seidel convergir mais rápido na maioria dos casos.

Interpolação de Lagrange — implementação completa (backend e frontend)
Implementamos o método de Interpolação de Lagrange do zero, tanto no backend quanto no frontend, com os 4 modos de detalhamento.

No backend, criamos a pasta methods/interpolacao/ com o arquivo lagrange.py. O algoritmo recebe uma lista de n+1 pontos $[(x_0,y_0), ..., (x□,y□)]$ e um valor x_eval, e retorna o valor interpolado P(x_eval), o grau do polinômio (n−1) e os detalhes de cada polinômio-base $L_i(x)$: numerador, denominador, o próprio L_i e o termo $y_i \cdot L_i$. O backend também valida se há valores duplicados de x, retornando um erro descritivo caso exista. No main.py, adicionamos o schema EntradaLagrange e o endpoint POST /lagrange.

No frontend, criamos:

- hooks/useLagrange.js — hook de requisição seguindo o mesmo padrão dos outros hooks do projeto
- pages/Lagrange.jsx — página com seletor de 2 a 6 pontos, tabela de entrada (x_i, y_i) e campo para o valor x_eval a interpolar

No modo Básico aparece apenas P(x_eval). No modo Intermediário e Completo é exibida uma tabela com as colunas: i, x_i, y_i, Numerador, Denominador, $L_i(x)$ e $y_i \cdot L_i(x)$, com uma linha de soma ao final mostrando P(x_eval).

Para o modo Interativo, criamos o arquivo components/interpolacao/LagrangeInterativo.jsx. O componente organiza o cálculo em fases: uma fase para cada polinômio-base $L_i(x)$ e uma fase final para a soma. Cada fase L_i tem 4 subpassos:

- Calcular o numerador: Π_{j≠i}(x_eval − x□)
- Calcular o denominador: Π_{j≠i}(x_i − x□)
- Calcular L_i(x_eval) = Numerador ÷ Denominador
- Calcular o $Termo_i$ = y_i × L_i(x_eval)

A fase final pede ao aluno que some todos os termos para obter P(x_eval). Uma mini-tabela lateral mostra todas as linhas com os valores sendo revelados progressivamente conforme o aluno avança nos subpassos. Também atualizamos o App.js para importar e renderizar a nova página Lagrange quando o usuário selecionar esse método no menu superior.